

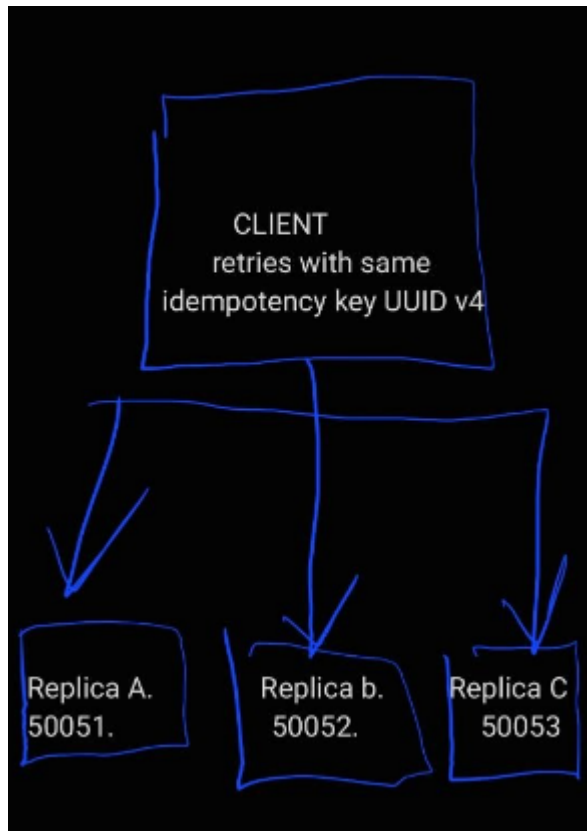
1. Requirements under test

This report verifies a three-stage replicated counter service: a gRPC single node with idempotent retries (Part A), Lamport-clock instrumentation (Part B), and a three-replica deployment with majority-acknowledged writes (Part C). Each requirement from the assignment is mapped to an automated test or a captured log trace; no claim in this report is made without such evidence.

Task	Requirement	Verifying evidence
A2	Increment applies its delta; retries with the same key must not reapply it	test_increment_applies_delta, test_duplicate_key_not_reapplied, test_retry_storm_same_key
A2	Get on unknown counter returns found=false	test_get_missing_counter
A2	Timeout + retry is safe (counter moves once)	test_retry_after_timeout_is_safe, test_induced_timeout_then_retry_moves_once
A3	Concurrent increments must not be lost	test_concurrent_increments_exact, test_counter_id_isolation
B1	SEND/RECV/APPLY logged with Lamport values	logs/*.log (Appendix A/B)
C1	Commit iff majority acks; one-replica failure tolerated	test_majority_commit_two_acks, test_no_commit_below_majority, test_replica_crash_mid_request_still_commits
C1	Duplicate suppression per replica; convergence	test_duplicate_request_returns_was_duplicate, test_replicas_converge
C4	Latency of single-replica vs quorum path	benchmark.py (Section 5)

Figure 1 shows the deployment topology this report validates.

Figure 1. Client fans each Increment out to three replicas; commit reported when 2 of 3 acknowledge.



2. Test environment

All tests ran on a single Windows machine over loopback, in a Python virtual environment. Package versions are recorded in Appendix C.

Component	Value
OS	Windows 11
Python	CPython 3.12.2
grpcio / grpcio-tools	1.84.0 / 1.84.0
protobuf / pytest	7.36.2 / 9.1.1
Network	loopback 127.0.0.1 only

Server executor	ThreadPoolExecutor(max_workers=8)
-----------------	-----------------------------------

3. Test-case table

Thirteen tests run automatically from pytest. Eight are mandated by the assignment; five are edge cases. All tests start their own servers on ephemeral ports, so pytest runs green from a clean checkout.

ID	Purpose	Expected	Actual	Result
T1	test_increment_applies_delta	5 then 8	5 then 8	PASS
T2	test_duplicate_key_not_reapplied	5, was_duplicate=true	5, true	PASS
T3	test_concurrent_increments_exact	exactly 2000	2000	PASS
T4	test_get_missing_counter	found=false	false	PASS
T5	test_retry_after_timeout_is_safe	value = 5, dup	5, true	PASS
T6	test_majority_commit_two_acks	2 acks → commit	2 acks	PASS
T7	test_no_commit_below_majority	$1 < 2 \rightarrow \text{fail}$	$1 < 2$	PASS
T8	test_replicas_converge	[3,3,3]	[3,3,3]	PASS
T9	test_replica_crash_mid_request_still_commits	commit with 2/3	2 acks	PASS

T10	test_duplicate_request_returns_was_duplicate	value stays	yes	PASS
T11	test_induced_timeout_then_retry_moves_once	value = 1	1, true	PASS
T12	test_counter_id_isolation	alpha=10, beta=5	10, 5	PASS
T13	test_retry_storm_same_key	1 apply, 19 dup	1, 19	PASS

Result: 13 passed in 3.81 s.

```

PS C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2> .venv\Scripts\activate
(.venv) PS C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2> pytest -v
===== test session starts =====
platform win32 -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2
collected 13 items

test_counter.py::test_increment_applies_delta PASSED [ 7%]
test_counter.py::test_duplicate_key_not_reapplied PASSED [ 15%]
test_counter.py::test_concurrent_increments_exact PASSED [ 23%]
test_counter.py::test_get_missing_counter PASSED [ 30%]
test_counter.py::test_retry_after_timeout_is_safe PASSED [ 38%]
test_counter.py::test_majority_commit_two_acks PASSED [ 46%]
test_counter.py::test_no_commit_below_majority PASSED [ 53%]
test_counter.py::test_replicas_converge PASSED [ 61%]
test_failures.py::test_replica_crash_mid_request_still_commits PASSED [ 69%]
test_failures.py::test_duplicate_request_returns_was_duplicate PASSED [ 76%]
test_failures.py::test_induced_timeout_then_retry_moves_once PASSED [ 84%]
test_failures.py::test_counter_id_isolation PASSED [ 92%]
test_failures.py::test_retry_storm_same_key PASSED [100%]

===== 13 passed in 3.81s =====
(.venv) PS C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2>

```

4. Failure-injection results

Each scenario injects a fault inside the test and asserts an externally observable invariant.

Scenario	Fault	Invariant	Test
Replica crash mid-request	Stop replica C	Commit with 2/3 acks	test_replica_crash_mid_request_still_commits

Request duplication	Same key twice	was_duplicate=true, value moves once	test_duplicate_request_ returns_was_duplicate
Induced timeout + retry	1 ms deadline + retry	Counter moves exactly once	test_induced_timeout_t hen_retry_moves_once
Counter isolation	Two counters	Independent values	test_counter_id_isolati on
Retry storm	20× same key	1 apply, 19 duplicates	test_retry_storm_same _key

The key observation: a failed write is not always a no-op. When only one replica acknowledges, that replica still applies the delta locally even though the client reports failure the partial-application anomaly shown in Figure 3. Raft avoids it by agreeing on the log entry before any replica mutates state; this assignment deliberately omits that mechanism.

```
(.venv) PS C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2> pytest -v test_failures.py
===== test session starts =====
platform win32 -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2
collected 5 items

test_failures.py::test_replica_crash_mid_request_still_commits PASSED [ 20%]
test_failures.py::test_duplicate_request_returns_was_duplicate PASSED [ 40%]
test_failures.py::test_induced_timeout_then_retry_moves_once PASSED [ 60%]
test_failures.py::test_counter_id_isolation PASSED [ 80%]
test_failures.py::test_retry_storm_same_key PASSED [100%]

===== 5 passed in 1.19s =====
(.venv) PS C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2>
```

5. Performance

benchmark.py measures latency for the four configurations required by the assignment, at least 2000 requests each. $p95 = \text{sorted}[\text{ceil}(0.95 \cdot n) - 1]$.

Configuration	Median (ms)	p95 (ms)	Requests
Single replica, 1 client	0.874	1.671	2000
Single replica, 16 clients	9.510	14.853	2000
Quorum (3 replicas), 1 client	3.149	4.452	2000

Quorum (3 replicas), 16 clients	55.648	79.784	2000
---------------------------------	--------	--------	------

The quorum path adds latency in three ways: each write is fanned out to three channels (three HTTP/2 round trips), the client waits for the slowest replica in the majority rather than the fastest, and at 16 clients there are 48 simultaneous channels competing for CPU on loopback. The 55 ms median in the last row is dominated by that context switching. For a like-counter workload this is acceptable: increments are infrequent, and the extra cost buys tolerance of one replica crash without losing committed writes.

```

===== 5 passed in 1.19s =====
(.venv) PS C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2> python benchmark.py
Running: Single replica, 1 client ...
Running: Single replica, 16 clients ...
Running: Quorum (3 replicas), 1 client ...
Running: Quorum (3 replicas), 16 clients ...

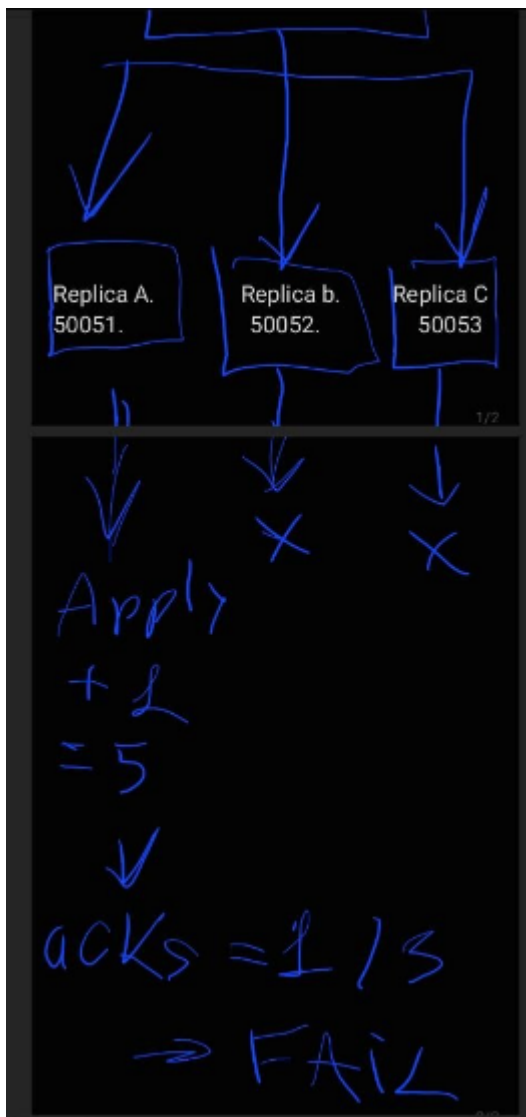
Configuration                Median (ms)    p95 (ms)    Requests
-----
Single replica, 1 client      0.597         1.206       2000
Single replica, 16 clients    8.221         9.663       2000
Quorum (3 replicas), 1 client 3.175         4.901       2000
Quorum (3 replicas), 16 clients 57.305       72.878       2000
(.venv) PS C:\Users\Asus\Desktop\Labs2-5 and Midterm\dcc-assignment2>

```

6. Known limitations

Two honest limitations apply. First, replicas do not share a log and there is no leader election, so a restarted replica returns with empty state and diverges from peers until new quorum writes overwrite the affected counters. Second, the deduplication store is in memory only, so a crashed replica loses its `_seen` map and will reapply a retried request as a fresh mutation, defeating idempotency across restarts. The smallest fix for both is to persist counters and the deduplication map to a write-ahead log and replay it on startup the mechanism Raft introduces in Week 6.

A third limitation is the partial-application anomaly shown in Figure 3.



Reflection questions

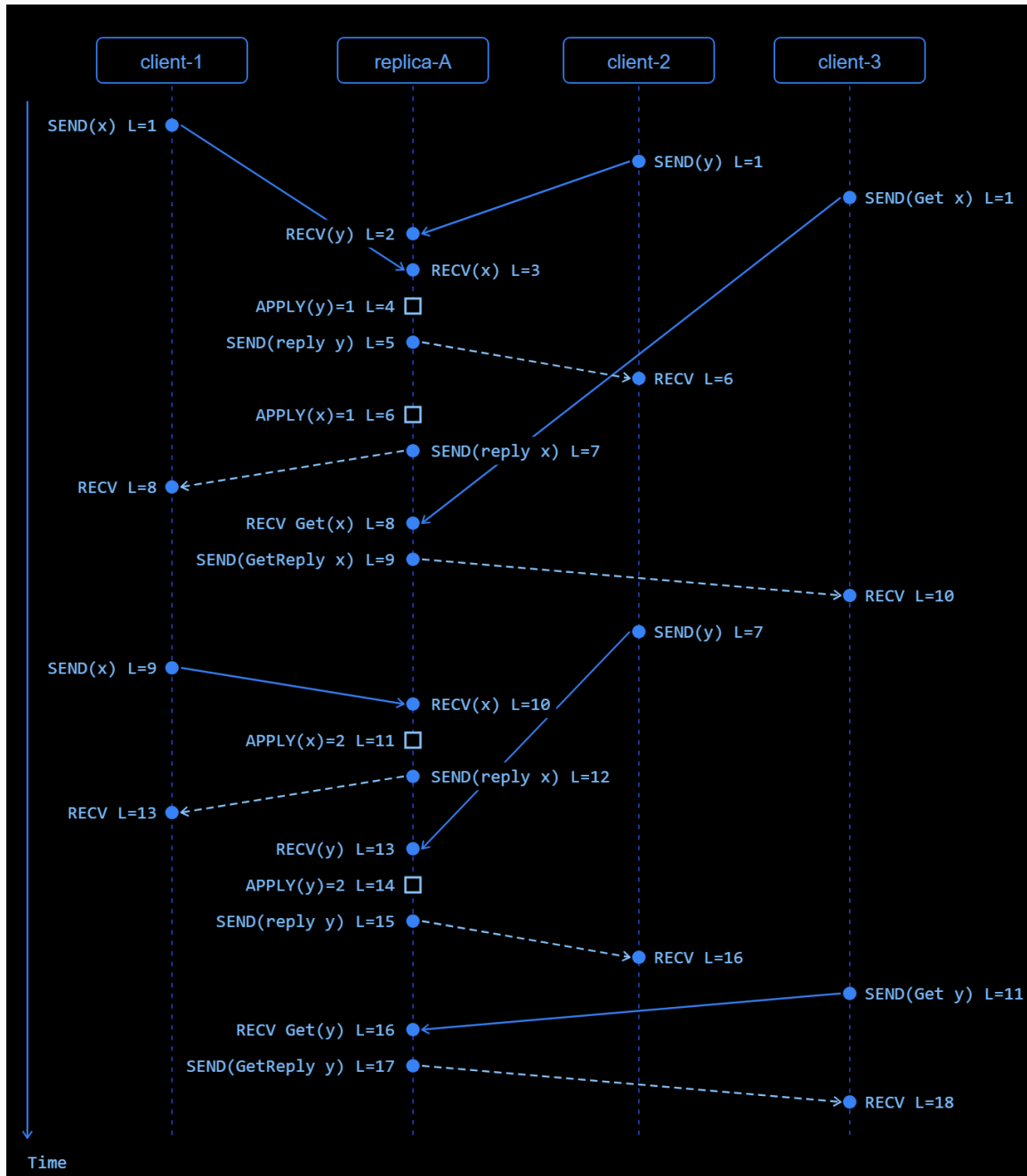
Q1 timeout ambiguity. After a timeout the client cannot tell whether the request was lost or the reply was lost; both produce the same symptom before the deadline. Idempotency makes this harmless: the client reuses the same key on every retry, and the server stores the result under that key in a deduplication map guarded by the same lock as the counter. A retry that reaches a server which already applied the request returns the stored value with `was_duplicate=true` and does not move the counter again.

Q2 Lamport values and concurrency. Two concurrent events can carry different Lamport values because each process keeps its own counter. The rule guarantees only $a \rightarrow b \Rightarrow L(a) < L(b)$. The converse does not hold, so $L(a) < L(b)$ may mean a happened first, or may mean the two events are concurrent and their counters simply started at different points. The smaller value cannot establish causality.

Q3 failure tolerance. Three replicas tolerate the failure of any one replica: two remain, majority is preserved, committed writes are safe and new writes still commit. Losing two replicas preserves safety (already-committed values remain on the survivor) but breaks liveness no new write can commit until a second replica returns. This matches the Week 5 safety/liveness distinction.

Q4 surviving restarts. The smallest change is to persist counters and the deduplication map to a write-ahead log and replay it on startup before accepting requests. This preserves committed writes and idempotency keys across crashes. It anticipates Raft's replicated log from Week 6, where each replica's state is a deterministic function of an ordered log of committed entries.

Appendix A Ordering analysis (Task B2)



Causal pair 1. client-1 SEND Increment L=1 → replica-A RECV L=2 (received L=1); the replica advanced to $\max(1,1)+1 = 2$. Causal pair 2. replica-A APPLY L=3 counter=x → 1 → client-1 RECV IncrementReply L=5 (received L=4); the client advanced to $\max(1,4)+1 = 5$. Both pairs are linked by a message edge, so causality is provable from the trace.

Concurrent pair. The two SEND Increment events from client 1 and client 2 in this run both carry $L=1$ on their own processes. Even if they carried different values, that difference would not establish ordering, because Lamport clocks only preserve causality in one direction. Neither event can reach the other through local order or messages, so they are concurrent.

Limitation of Lamport clocks. They cannot distinguish “ $a \rightarrow b$ ” from “ a and b are concurrent but $L(a) < L(b)$ ”. Vector clocks remove this by carrying one component per process, so concurrency becomes a direct componentwise comparison.

Appendix B Log excerpt (30 lines)

```
replica-A listening on port 50051
replica-A RECV Increment L=2 counter=x delta=1 (received L=1)
replica-A APPLY L=3 counter=x -> 1
replica-A SEND IncrementReply L=4 (new_value=1, duplicate=false)
replica-A RECV Increment L=5 counter=x delta=1 (received L=1)
replica-A APPLY L=6 counter=x -> 2
replica-A SEND IncrementReply L=7 (new_value=2, duplicate=false)
replica-A RECV Increment L=8 counter=y delta=1 (received L=1)
replica-A APPLY L=9 counter=y -> 1
replica-A SEND IncrementReply L=10 (new_value=1, duplicate=false)
replica-A RECV Increment L=11 counter=y delta=1 (received L=1)
replica-A APPLY L=12 counter=y -> 2
replica-A SEND IncrementReply L=13 (new_value=2, duplicate=false)
replica-A RECV Get L=14 counter=x (received L=1)
replica-A SEND GetReply L=15 (value=2, found=True)
replica-A RECV Get L=16 counter=y (received L=1)
replica-A SEND GetReply L=17 (value=2, found=True)

client-1 SEND Increment L=1 counter=x delta=1
client-1 RECV IncrementReply L=5 (new_value=1, duplicate=False, received L=4)
client-1 SEND Increment L=1 counter=x delta=1
client-1 RECV IncrementReply L=8 (new_value=2, duplicate=False, received L=7)

client-2 SEND Increment L=1 counter=y delta=1
client-2 RECV IncrementReply L=11 (new_value=1, duplicate=False, received L=10)
client-2 SEND Increment L=1 counter=y delta=1
client-2 RECV IncrementReply L=14 (new_value=2, duplicate=False, received L=13)

client-3 SEND Get L=1 counter=x

client-3 RECV GetReply L=16 (value=2, found=True, received L=15)
```

Appendix C pip freeze

```
grpcio==1.84.0
grpcio-tools==1.84.0
```

protobuf==7.36.2

pytest==9.1.1

colorama==0.4.6

iniconfig==2.3.0

packaging==26.3

pluggy==1.6.0

Pygments==2.21.0

setuptools==84.0.0

typing_extensions==4.16.0